Introdução ao .NET Core

Do Console às Web APIs

Prof. Geovani | 17/03/2026 - Aula 6

Marcos Importantes

Data	Evento	Detalhes
03/02	■ Aula 1	Apresentação (Realizada)
17/03	■ Entrega 1	Proposta do Projeto
05/05	■ AP (30%)	Avaliação Parcial
26/05	■ Entrega 2	Modelagem do BD
02/06	■ Aula 16	JWT - ÚLTIMA AULA
09/06	■ AF (30%)	APRESENTAÇÃO FINAL
16/06	■ AI (40%)	Avaliação Interdisciplinar
23/06	■ AIS	Substitutiva (se aplicável)
30/06	■ Aula 20	Encerramento

Atenção ao nosso cronograma

Aula	Data	Conteúdo Principal	Evento/Entrega
1	03/02	Apresentação da Disciplina	
2	10/02	Fundamentos C# - Parte 1	-
-	17/02	CARNAVAL - Sem aula	Feriado
3	24/02	Fundamentos C# Parte 2 + Coleções	-
4	03/03	OOP - Parte 1	-
~~5~~	~~10/03~~	~~OOP - Parte 2~~	~~-~~
6	17/03	.NET Core e ASP.NET Core	Entrega Proposta de Projeto
7	24/03	SOLID + Design Patterns	-
8	31/03	Clean Architecture	-
-	03-04/04	Paixão de Cristo - Suspensão	Feriado
9	07/04	HTTP e REST	-
10	14/04	Construindo APIs REST	-
-	21/04	Tiradentes - Feriado	Feriado
11	28/04	Middleware e Filtros	-
12	05/05	AVALIAÇÃO PARCIAL (AP)	AP - 30%
13	12/05	PostgreSQL + EF Core	-
14	19/05	Repository Pattern + BD	-
15	26/05	Queries Avançadas	Entrega da Arquitetura de Software do projeto
16	**02/06**	**AUTENTICAÇÃO JWT**	**FIM CONTEÚDO**
-	04-06/06	Corpus Christi - Suspensão	Feriado
17	09/06	APRESENTAÇÃO FINAL (AF)	AF - 30%
18	16/06	AVALIAÇÃO INTERDISCIPLINAR (AI)	AI - 40%
19	23/06	Avaliação Substitutiva (AIS)	AIS
20	30/06	Encerramento + Revisão	Fim Semestre

Objetivos de Hoje

🎯 Entender arquitetura do .NET Core
🎯 Conhecer o ASP.NET Core
🎯 Criar primeira Web API
🎯 Dominar estrutura de projetos Web API
🎯 Entender Middlewares e Pipeline
🎯 Trabalhar com Dependency Injection
🎯 Configurar ambientes (Dev, Staging, Prod)

Revisão: Aula 5 - OOP Parte 2

- ✅ Herança (base, sealed)
- ✅ Polimorfismo (virtual, override)
- ✅ Classes Abstratas (abstract)
- ✅ Interfaces (múltiplas)
- ✅ Casting (is, as)
- ✅ Pattern Matching
- ✅ Abstrata vs Interface

O que é .NET Core?

Arquitetura e Fundamentos

História do .NET - Evolução

📅 2002: .NET Framework 1.0

- Apenas Windows
- Framework monolítico

📅 2016: .NET Core 1.0

- Cross-platform (Windows, Linux, macOS)
- Open-source
- Modular e leve

📅 2020: .NET 5 (unificação)

- .NET Framework + .NET Core = .NET 5+
- Versão única multiplataforma

📅 2024: .NET 8 (LTS) ← USAREMOS ESSA!

- Performance otimizada
- Suporte de longo prazo

.NET Framework vs .NET Core vs .NET 5+

Característica	.NET Framework	.NET Core	.NET 5+
Plataforma	Apenas Windows	Cross-platform	Cross-platform
Open Source	❌ Não	✅ Sim	✅ Sim
Performance	Boa	Excelente	Excelente
Arquitetura	Monolítica	Modular	Modular
Versão Atual	4.8 (final)	Descontinuado	.NET 8 (LTS)
Futuro	Manutenção apenas	Migrado para .NET 5+	✅ Ativo
Deploy	IIS (Windows)	Qualquer servidor	Qualquer servidor

Arquitetura do .NET Core

📦 Componentes Principais:

1️⃣ CoreCLR (Runtime)

- CLR = Common Language Runtime
- Executa código gerenciado
- Garbage Collector (GC)
- JIT Compiler (Just-In-Time)

2️⃣ CoreFX (Libraries)

- Bibliotecas base (System.*)
- Collections, IO, Networking, etc
- APIs multiplataforma

3️⃣ SDK (Software Development Kit)

- Compilador (Roslyn)
- CLI (dotnet command)
- Templates de projeto

Compilação e Execução - Como Funciona

📝 Código C# (.cs)

↓

🔨 Compilador Roslyn

↓

📦 IL Code (Intermediate Language)

Código intermediário, não nativo

Arquivo .dll ou .exe

↓

⚙️ JIT Compiler (em runtime)

Converte IL → código nativo da máquina

↓

🚀 Execução no CoreCLR

💡 Benefício: Escrever uma vez, rodar em qualquer SO!

IL é portável, JIT adapta ao SO/hardware

ASP.NET Core

Framework Web da Microsoft

O que é ASP.NET Core?

🌐 ASP.NET Core:

Framework web moderno e cross-platform

para construir aplicações web e APIs

🎯 Tipos de Aplicações:

- Web APIs (REST/GraphQL) ← FOCO DO CURSO
- MVC (Model-View-Controller)
- Razor Pages (páginas dinâmicas)
- Blazor (SPA com C#)
- SignalR (real-time)

💪 Por que usar ASP.NET Core?

✅ Alta performance (mais rápido que Node.js!)

✅ Cross-platform

✅ Open-source e gratuito

✅ Seguro e escalável

✅ Comunidade ativa

Criando Primeiro Projeto Web API

```
// No terminal/prompt
dotnet new webapi -n MinhaAPI

// Explicação:
// dotnet new = criar novo projeto
// webapi = template de Web API
// -n MinhaAPI = nome do projeto

// Entrar na pasta
cd MinhaAPI

// Restaurar dependências (NuGet)
dotnet restore

// Compilar
dotnet build

// Executar
dotnet run

// Ou executar com watch (Hot Reload)
dotnet watch run

// Abrir no navegador:
// https://localhost:7XXX/swagger
```

Estrutura de um Projeto Web API

```
MinhaAPI/
│
├── Controllers/          // ← Endpoints da API
│   └── WeatherForecastController.cs
│
├── Properties/
│   └── launchSettings.json  // Configurações de execução
│
├── appsettings.json      // ← Configurações gerais
├── appsettings.Development.json  // ← Config de desenvolvimento
│
├── Program.cs            // ← PONTO DE ENTRADA (main)
│
├── WeatherForecast.cs    // Model de exemplo
│
├── MinhaAPI.csproj       // Arquivo do projeto
├── MinhaAPI.http         // Testes HTTP
└── .gitignore

// IMPORTANTE:
// • Program.cs = configuração da aplicação
// • Controllers/ = seus endpoints
// • appsettings.json = configurações
```

Program.cs - Coração da Aplicação

```
var builder = WebApplication.CreateBuilder(args);

// 1. CONFIGURAR SERVIÇOS (Dependency Injection)
builder.Services.AddControllers();  // Adiciona suporte a Controllers
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();   // Documentação automática

// 2. CONSTRUIR A APLICAÇÃO
var app = builder.Build();

// 3. CONFIGURAR MIDDLEWARES (Pipeline HTTP)
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();      // Habilita Swagger em Dev
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();  // Redireciona HTTP → HTTPS
app.UseAuthorization();     // Middleware de autorização
app.MapControllers();       // Mapeia os Controllers

// 4. EXECUTAR
app.Run();

// ⚠️ ORDEM DOS MIDDLEWARES IMPORTA!
```

Criando Primeiro Endpoint

```
// Controllers/HelloController.cs
using Microsoft.AspNetCore.Mvc;

[ApiController]  // Indica que é um controller de API
[Route("api/[controller]")]  // Rota: /api/hello
public class HelloController : ControllerBase
{
    // GET: api/hello
    [HttpGet]
    public ActionResult<string> Get()
    {
        return Ok("Olá, mundo! Minha primeira API!");
    }

    // GET: api/hello/nome
    [HttpGet("{nome}")]
    public ActionResult<string> GetComNome(string nome)
    {
        return Ok($"Olá, {nome}! Bem-vindo à API!");
    }
}

// Testar:
// GET https://localhost:7XXX/api/hello
// GET https://localhost:7XXX/api/hello/Geovani
```

Middlewares

Pipeline de Requisições HTTP

Middlewares - Conceito Fundamental

📌 Definição:

Componentes que formam o PIPELINE de requisição

Cada middleware processa a requisição e decide:

- Passar para o próximo (next)
- Retornar resposta imediata

🔄 Fluxo:

Request → MW1 → MW2 → MW3 → Endpoint

Response ← MW1 ← MW2 ← MW3 ← Endpoint

⚠️ ORDEM IMPORTA!

Executam na ordem que são registrados

💡 Analogia:

Filtros de água em série

Cada filtro faz uma coisa específica

Middlewares Built-in do ASP.NET Core

```
// Ordem RECOMENDADA dos middlewares:

app.UseHttpsRedirection();  // 1. Redireciona HTTP → HTTPS

app.UseStaticFiles();       // 2. Serve arquivos estáticos (CSS, JS, imgs)

app.UseRouting();           // 3. Roteamento de requisições

app.UseCors();              // 4. CORS (Cross-Origin)

app.UseAuthentication();    // 5. Autenticação (quem é?)

app.UseAuthorization();     // 6. Autorização (pode fazer?)

app.MapControllers();       // 7. Mapeia endpoints dos Controllers

app.Run();                  // 8. Executa a aplicação

// ⚠️ IMPORTANTE:
// UseAuthentication ANTES de UseAuthorization!
// UseRouting ANTES de UseEndpoints/MapControllers!
```

Dependency Injection

Injeção de Dependências

Dependency Injection (DI) - Conceito

📌 Problema SEM DI:

```
public class ProdutoController
{
    private ProdutoRepository _repo = new ProdutoRepository();
    // ❌ Acoplamento forte!
    // ❌ Dificulta testes
}
```

✅ Solução COM DI:

```
public class ProdutoController
{
    private readonly IProdutoRepository _repo;

    public ProdutoController(IProdutoRepository repo)
    {
        _repo = repo;  // Injetado!
    }
}
```

💡 DI = Framework cria e injeta dependências
Controller não cria, apenas RECEBE

Container de DI do ASP.NET Core

```
// No Program.cs - Registrar serviços no CONTAINER

// 1. TRANSIENT: Nova instância SEMPRE
builder.Services.AddTransient<IProdutoService, ProdutoService>();
// Cada requisição = nova instância
// Útil para serviços leves/stateless

// 2. SCOPED: Uma instância por REQUISIÇÃO HTTP
builder.Services.AddScoped<IProdutoRepository, ProdutoRepository>();
// Mesma instância durante toda a requisição
// ✅ RECOMENDADO para repositórios/DbContext

// 3. SINGLETON: Uma instância para TODA APLICAÇÃO
builder.Services.AddSingleton<IConfiguration>(builder.Configuration);
// Criada na primeira vez, reutilizada sempre
// ⚠️ Cuidado com estado compartilhado!

// Depois, ASP.NET Core injeta automaticamente nos construtores!
```

Lifetimes - Transient vs Scoped vs Singleton

Característica	Transient	Scoped	Singleton
Criação	Sempre nova	Uma por requisição	Uma para sempre
Compartilhamento	Não compartilha	Compartilha na req	Compartilha global
Performance	Mais lento	Balanceado	Mais rápido
Memória	Mais uso	Médio uso	Menos uso
Thread-safe?	Não precisa	Não precisa	⚠️ Precisa!
Uso típico	Services leves	Repositories, DbContext	Config, Cache
Quando usar	Stateless simples	✅ Padrão para APIs	Dados globais

Configurações

appsettings.json e Ambientes

appsettings.json - Configurações da Aplicação

```
// appsettings.json - Configurações GERAIS
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",

  // Suas configurações customizadas:
  "ConnectionStrings": {
    "DefaultConnection": "Host=localhost;Database=meudb;Username=postgres;Password=senha"
  },
  "AppSettings": {
    "NomeApp": "Minha API",
    "VersaoAPI": "1.0",
    "MaxItensPorPagina": 50
  }
}

// ✅ Vantagens:
// • Separar config do código
// • Fácil alterar sem recompilar
// • Diferentes configs por ambiente
```

Ambientes: Development, Staging, Production

🌍 Ambientes (Environments):

📝 Development (Desenvolvimento)
- Máquina do desenvolvedor
- Logs detalhados, Swagger ativo
- appsettings.Development.json

🧪 Staging (Homologação)
- Ambiente de testes pré-produção
- Configuração similar a produção
- appsettings.Staging.json

🚀 Production (Produção)
- Servidor real, usuários reais
- Logs mínimos, sem Swagger
- appsettings.Production.json

💡 ASP.NET Core escolhe automaticamente
baseado em variável ASPNETCORE_ENVIRONMENT

Lendo Configurações no Código

```
// No Controller - Injetar IConfiguration

[ApiController]
[Route("api/[controller]")]
public class ConfigController : ControllerBase
{
    private readonly IConfiguration _configuration;

    public ConfigController(IConfiguration configuration)
    {
        _configuration = configuration;  // Injetado por DI
    }

    [HttpGet("info")]
    public ActionResult GetInfo()
    {
        var nomeApp = _configuration["AppSettings:NomeApp"];
        var versao = _configuration["AppSettings:VersaoAPI"];
        var connectionString = _configuration
            .GetConnectionString("DefaultConnection");

        return Ok(new {
            Nome = nomeApp,
            Versao = versao,
            Ambiente = _configuration["ASPNETCORE_ENVIRONMENT"]
        });
    }
}
```

Estrutura Completa

Juntando Tudo

Fluxo Completo de uma Requisição

📨 REQUISIÇÃO HTTP
GET /api/produtos
↓
🌐 Middlewares (pipeline)
UseHttpsRedirection → UseRouting → UseAuthorization
↓
🎯 Roteamento
Identifica Controller e Action corretos
↓
💉 Dependency Injection
Cria instâncias e injeta dependências
↓
🎮 Controller
ProdutosController.Get() executa
↓
📦 Service/Repository
Lógica de negócio, acesso a dados
↓
↩️ RESPOSTA HTTP
200 OK + JSON

Exemplo Completo - API Produtos

```
// 1. Interface
public interface IProdutoRepository
{
    Task<List<Produto>> GetAllAsync();
}

// 2. Implementação
public class ProdutoRepository : IProdutoRepository
{
    public async Task<List<Produto>> GetAllAsync()
    {
        // Simula busca no banco
        return new List<Produto>
        {
            new Produto { Id = 1, Nome = "Notebook", Preco = 3000 },
            new Produto { Id = 2, Nome = "Mouse", Preco = 50 }
        };
    }
}

// 3. Registrar DI (Program.cs)
builder.Services.AddScoped<IProdutoRepository, ProdutoRepository>();

// 4. Controller
[ApiController]
[Route("api/[controller]")]
public class ProdutosController : ControllerBase
{
    private readonly IProdutoRepository _repository;

    public ProdutosController(IProdutoRepository repository)
    {
        _repository = repository;  // Injetado!
    }

    [HttpGet]
    public async Task<ActionResult<List<Produto>>> Get()
    {
        var produtos = await _repository.GetAllAsync();
        return Ok(produtos);
    }
}
```

Hora de Praticar!

Criando sua primeira Web API

Exercício 1: API Hello World Completa

🎯 DESAFIO:
Criar API com 3 endpoints diferentes

📋 Requisitos:

1️⃣ Endpoint Simples
GET /api/hello
Retorna: "Olá, mundo!"

2️⃣ Endpoint com Parâmetro de Rota
GET /api/hello/{nome}
Retorna: "Olá, {nome}!"

3️⃣ Endpoint com Query String
GET /api/hello/cumprimento?nome=João&hora=18
Retorna: "Boa tarde, João!" (lógica baseada na hora)

✅ Teste todos no navegador ou Postman

Fim de Jogo

https://w.app/gil

Obrigado e até a próxima aula!